IA E LABORATORIO · PROGETTO CLIPS · A.A. 2025/26

# Tre agenti nella tana del Wumpus

Moduli, deduzione logica e certainty factors per "L'oro del Wumpus"

Alfredo De Donno · matricola 810700

Modulo del Prof. R. Micalizio

IL PROBLEMA

# Prima abbiamo letto il simulatore

**+100**

per l'oro, preso una volta sola

**±25**

per ogni dichiarazione di pozzo, giusta o sbagliata

**−100**

se si muore, o anche solo se finisce l'energia

Il costo dei passi **non entra nel premio**: consuma solo energia. Quindi conviene dichiarare i pozzi e prendere l'oro **per ultimo**.

wumpus.py, metodo WumpusWorld.step()

ARCHITETTURA

# Un passo = un giro dei moduli




**avvia-passo** scatta perché il simulatore **modifica** counter: è un fatto nuovo, e la rifrazione non lo blocca

**(focus …)** impila i moduli; ognuno esce quando la sua agenda è vuota

In DECISIONE ogni regola richiede **(not (decisione))**: ne scatta una sola, scelta per salience

AZIONE asserisce **un solo** fatto action e ricorda la mossa, per riconoscere gli urti

Bordo rosso: modulo presente solo nell'agente 3 · INFERENZA e DISTANZE: agenti 2 e 3

AGENTE 1 · PRUDENTE

# Sicuro solo ciò che vede da vicino

## Cosa sa fare

Una sola regola di sicurezza: niente brezza **e** niente puzza qui, quindi le 4 vicine sono sicure

Visita in profondità con la **pila** del cammino; torna indietro sui suoi passi

Prende l'oro appena lo vede; torna a casa se energia ≤ pila + 2

## Cosa non sa fare

Combinare informazioni di celle diverse

Dedurre dove sono pozzi e Wumpus, quindi niente +25

Cercare il cammino più breve

agenti/agente-base/agent.clp · moduli PERCEZIONE, DECISIONE, AZIONE

AGENTE 2 · LOGICO

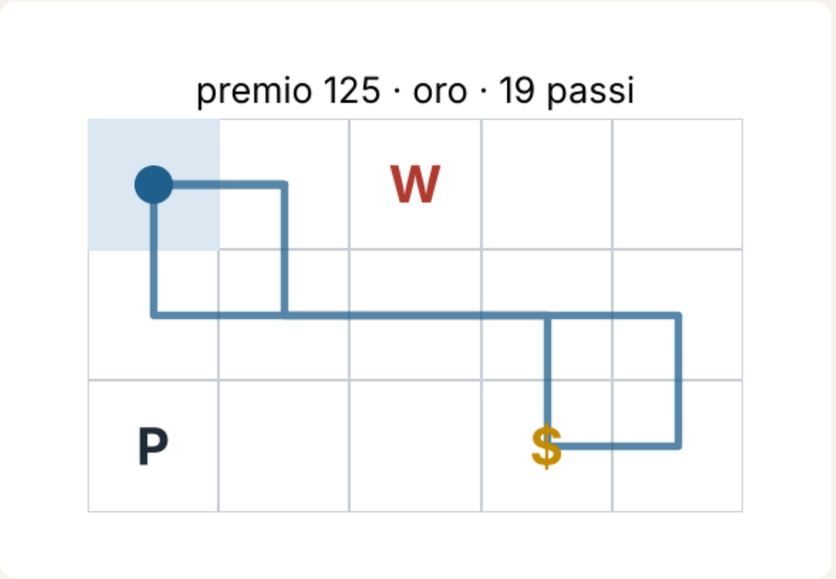
# Deduce come un detective

**sicura** = no-pozzo **e** no-Wumpus, anche se le due prove vengono da celle diverse

**Eliminazione**: una brezza con un solo candidato rimasto indica un pozzo certo, e si dichiara (+25)

**Un solo Wumpus**: dev'essere vicino a *tutte* le celle con puzza

Le **saliance** ordinano le fasi: genero i candidati, li scarto, poi concludo




m04: l'agente 1 resta bloccato a 0, il 2 fa 125

AGENTI 2 E 3 · MODULO DISTANZE

# Una BFS scritta a regole

```
(defrule DISTANZE::propaga-casa
  (dist (tipo casa) (x ?x) (y ?y) (d ?d))
  (visitata (x ?nx)
    (y ?ny&:(adiacenti ?x ?y ?nx ?ny)))
  (not (dist (tipo casa) (x ?nx) (y ?ny)))
  ⇒
  (assert (dist (tipo casa) (x ?nx) (y ?ny)
                (d (+ ?d 1)))))
```

Si parte dai bersagli (d = 0) e ogni regola propaga d+1; **accorcia** corregge i giri lunghi

Verso casa si passa **solo su celle visitate**: nessun muro nascosto lungo il ritorno

Un **gettone** (ricalcola) cancella i campi vecchi una volta sola; senza, ciclo infinito

Campi: casa, esplora, oro, dichiara, rischio

AGENTE 3 · STRATEGA

# Certainty factors e una strategia sul punteggio

## Modulo CERTEZZA

Brezza con k candidati: **1/k** a ciascuno; già spiegata da un pozzo certo: **0,2**

Combinazione MYCIN: **CF = a + b(1 – a)**

Pericolo = combinazione di CF pozzo e CF Wumpus

## Strategia

Dichiara anche i pozzi con **CF ≥ 0,75**, andandoci apposta

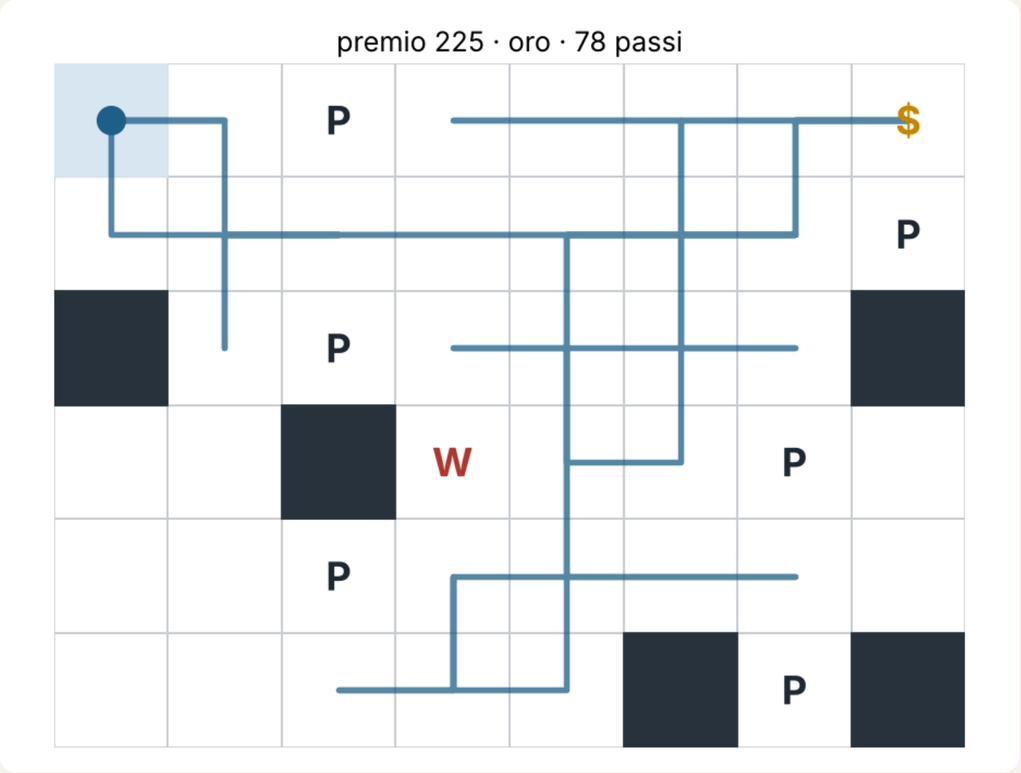
**L'oro per ultimo**, conservando l'energia per tornarci

**Rischio calcolato** nella cella meno pericolosa (≤ 0,3), solo se l'oro non è stato trovato

agenti/agente-3/agent.clp · parametri nella deffacts di MAIN

UNA PARTITA · MAPPA M01, LA NOSTRA

# Dedurre, dichiarare, rischiare poco

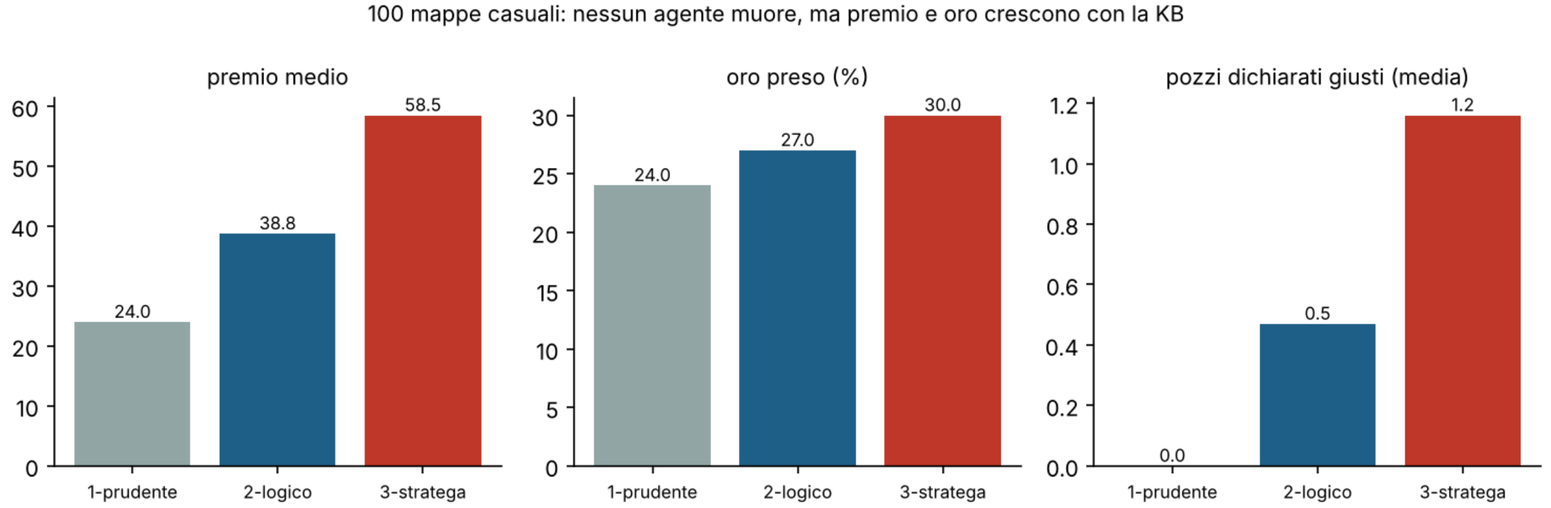



1. Deduce il pozzo (2,0) per eliminazione e lo dichiara: +25
2. Finite le celle sicure, entra in una cella con pericolo 0,2, perché la brezza era già spiegata. Sopravvive
3. Trova il Wumpus in (3,3) e altri pozzi certi; ne dichiara 5 in tutto, uno con CF 0,75
4. Solo alla fine va all'oro, lo prende e torna a casa

Stessa mappa: agente 1 → 0, agente 2 → 25, agente 3 → 225 · strumenti/disegna_partita.py

RISULTATI · 100 MAPPE CASUALI + 9 A TEMA

# Più conoscenza, più premio, zero morti




Premio medio **× 2,4** dal primo al terzo agente, con 0% di morti per tutti

L'oro cresce poco: il perché lo dicono **gli oracoli** (slide seguente)

Mappe a tema: l'agente 3 è **sempre ≥** del 2, e il 2 sempre ≥ dell'1

strumenti/esperimenti.py · risultati/riepilogo.csv, risultati_mappe.csv

ANALISI CRITICA · DUE ORACOLI

# La logica è completa: oltre si va solo rischiando

**84%**

mappe con l'oro raggiungibile per chi **vede tutta la mappa** (oracolo onnisciente)

**27%**

mappe con l'oro preso dall'agente 2 usando **solo la logica**

**0/100**

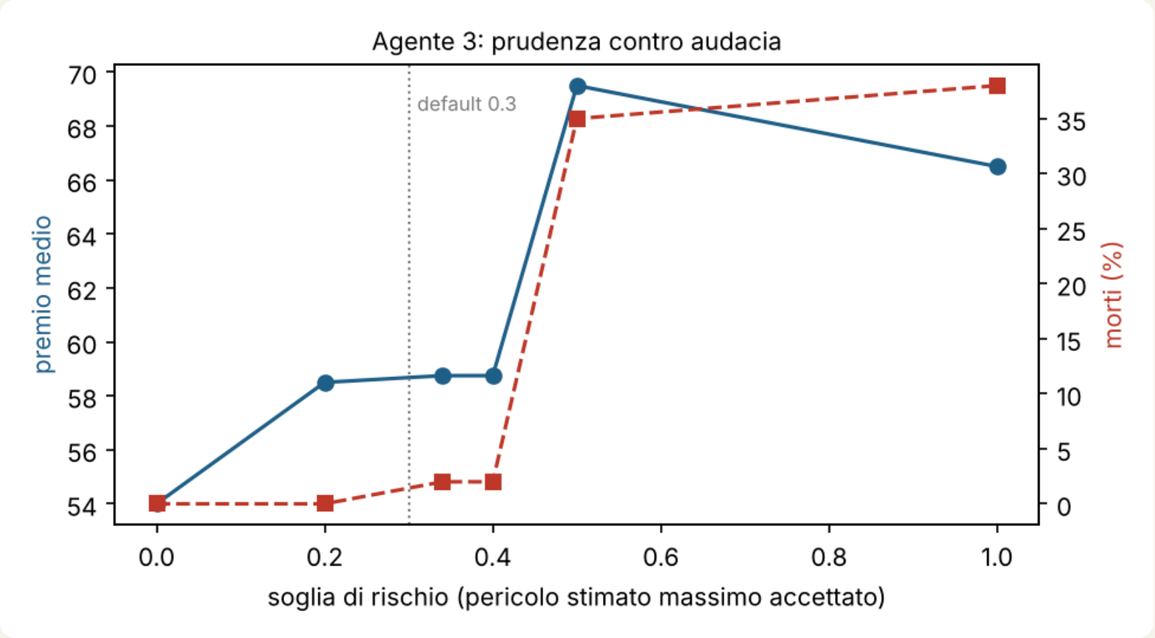
partite in cui l'agente 2 lascia inesplorata una cella **dimostrabilmente sicura** (agente 1: 7/100)

L'oracolo logico rifà il ragionamento con un **model checking completo**, indipendente da CLIPS. Le nostre regole deducono tutto il deducibile, quindi il divario tra 27% e 84% si colma solo rischiando: è il lavoro dell'agente 3.

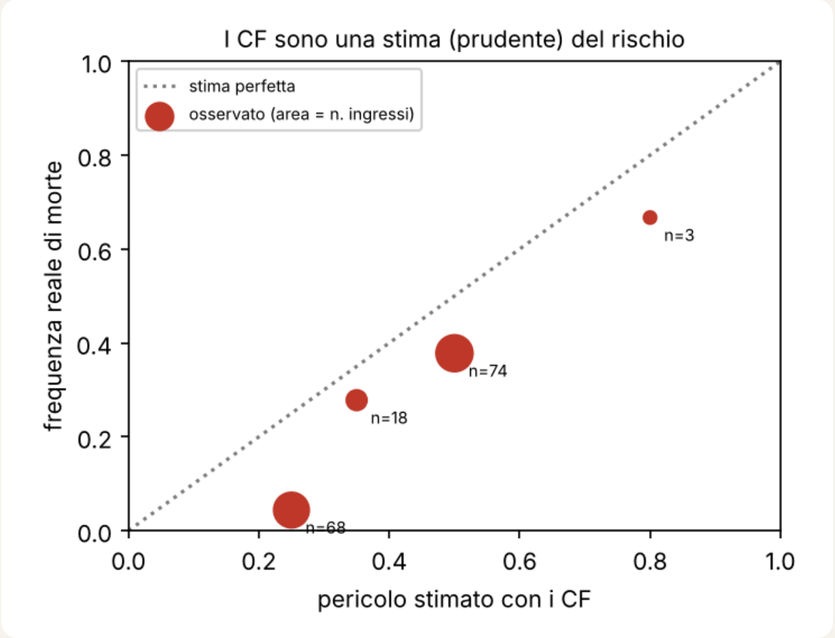
strumenti/oracolo.py · strumenti/oracolo_logico.py (109 celle sicure verificate, 0 errori)

ANALISI CRITICA

# Prudenza contro audacia




Soglia 0,5: premio 69,5 ma **35% di morti**. Abbiamo scelto 0,3: 58,5 e nessuna morte




I CF non sono probabilità, ma la stima è **monotona**: bastano per decidere

strumenti/esperimenti.py --soglie · strumenti/calibrazione_cf.py (163 ingressi a rischio)

PROBLEMI INDIVIDUATI

# Cosa ci ha insegnato il debug

## Nel simulatore

Al passo 0 non arriva nessun percept: si parte con **grab**, l'unica azione innocua

Senza action viene eseguito **right**: c'è una regola "paracadute"

Lo stesso pozzo dà +25 a ogni dichiarazione: falla **non sfruttata**

Serve Python ≥ 3.12 e va lanciato dalla cartella simulatore

## Nel nostro agente 3

Su una mappa con poca energia finiva l'energia: –100

Causa: una funzione dentro un pattern viene valutata **al momento del matching**, quando arriva il fatto energia, prima che le distanze del passo esistano

Soluzione: calcolare la riserva come **fatto**, alla fine di DISTANZE

Dettagli nel README di clips-wumpus

CONCLUSIONI

# L'intelligenza sta nelle regole

Stesso motore, stesso simulatore: cambia solo la KB, e il premio medio passa da **24** a **39** a **58,5**

I moduli separano il controllo dal dominio; i CF danno una stima del rischio economica e sufficiente

## Sviluppi futuri

Rete bayesiana al posto dei CF, per probabilità esatte

Soglia di rischio appresa (reinforcement learning)

A* invece della BFS a ondata

Codice, mappe e dati: repository astar-to-wumpus · clips-wumpus/